

Laboratory Exercise 07

Topics

1. Lists
2. Use the `for` loop to iterate through a list
3. Functions that make use of lists

Discussion

- A. What is a list?
- B. What are the differences between an item in a list, and its index?
- C. What information do you need on a list to scroll through it without making mistakes?

Exercises (in red = suggested ones to complete during the lab)

Part 1 – Elaborating lists

Goal: For each of the following exercises, write a Python program that responds to the above requests. Complete at least two exercises during the lab session, and the remaining ones at home.

07.1.1 Alternating sign sum. Write a program that receives as input a sequence of integers (terminated by a blank line), and that calculates the alternating sum of its elements. For example, if the program reads data `1 4 9 16 9 7 4 9 11`, it should calculate and display `1 - 4 + 9 - 16 + 9 - 7 + 4 - 9 + 11 = -2`. [P6.8]

[SUGGESTED] 07.1.2 List of random numbers. Write a program that initializes a list with ten random integers between `1` and `100` and then displays, on four successive lines:

- I. All elements of even **index**;
- II. All elements of even **value**;
- III. All elements in **reverse** order;
- IV. The first and last element. [P6.1]

07.1.3 Remove the minimum value. Write a function `remove_min(v)` that removes the minimum value from a list `v` without using the `min()` function or the `remove()` method. [P6.7]

07.1.4 Local maxima. Read a sequence of integers that ends with an empty line. Print the position of the local maxima (numbers greater than both the previous and the next value) if there are any, otherwise print the message `'There are no local maxima'`.

Extra: If there are at least two local maxima, locate the two closest local maxima and print their position.

[SUGGESTED] 07.1.5 The same elements. Write the function `same_set(a, b)` that checks if two lists contain the same elements, regardless of the order and ignoring the presence of duplicates. For example, the function must return `True` for the two lists `1 4 9 16 9 7 4 9 11` and `11 11 7 9 16 4 1`. The function must not modify lists that have been passed as parameters. [P6.12]

07.1.6 Ordered list. Write a program that generates a sequence of `20` random integer values between `0` and `99`, then displays the generated sequence, sorts it, and displays it again, ordered. Use the `sort()` method. [P6.17]

07.1.7 Sum without minimum. Write the function `sum_without_smallest(v)` that calculates the sum of all the values of a list `v`, excluding the minimum value. [P6.6]

Part 2 – Algorithms that make use of lists

Goal: For each of the following exercises, write a Python program that responds to the above requests. Complete at least two exercises during the lab session, and the remaining ones at home.

07.2.1 Measurement noise. Often the data collected during an experiment must be processed to remove some of the measurement noise. A simple approach to this problem is to replace each value in a list of values with the average of the value itself and the two adjacent values (or of the only adjacent value if the value is at either end of the list). Write a program that does this, without creating a second list. [P6.36]

07.2.2 Spacing. People who park their automobile in a parking row usually prefer to maximize the distance between the space they occupy and the spaces that are already occupied by other vehicles. They therefore tend to occupy the middle seat of the longest row of free seats available.

For example, consider the situation where ten places are free:

— — — — —

The first person to arrive will occupy, with his or her vehicle, a seat in the middle of the row:

— — — — X — — — —

The next person will place it in the middle of the longest row left free (i.e. the one on the left):

— X — X — — — —

Write a program that receives as input the number of parking spaces that make up the row and that, each time a new space is occupied according to the rule indicated, displays the row in the format shown above. *Tip:* Use a list of Boolean values to indicate whether a parking space is occupied or not. [P6.19]

[SUGGESTED] 07.2.3 Bulgarian solitaire. The game of Bulgarian Solitaire starts with 45 cards. The cards are randomly divided into a given number of piles with random sizes. For example, you can start with 5 piles of sizes 20, 5, 1, 9, and 10. Each turn, you subtract a card from each pile, and create a new pile made up of the cards subtracted from the starting piles. For example, the starting configuration would be transformed into piles of size 19, 4, 8, 9, and 5. The game ends when the piles are of size 1, 2, 3, 4, 5, 6, 7, 8 and 9 (you can prove that you always get such a configuration). Write a program that generates a random initial configuration and displays it, and then continues with the solitaire steps, displaying the configuration after each step, and stopping when the final solitaire configuration is reached. [P6.20]